MIT 6.006

13 October 2026 12:35

Lecture 1: Algorithms & Computations

Goal → 1) solve computational problems
2) prove correctness
3) argue efficiency

4) communication of problems

Correctness: Use Induction to prove correctness in recursion questions
Base Case: $n=1$ or $n=0$
Inductive hypothesis: assume the statement is true for some arbitrary natural number
Inductive step: using the hypothesis assumption, prove its true for $k+1$

Problem: possible inputs & outputs
↳ computational problem: binary relation b/w set of inputs & outputs

How to map with computer's memory?

RAM (Random access memory) – we assume computer can access any address in memory in constant time

memory is Byte addressable

Memory is conceptually → an arrangement of addresses

Add 001	Add 002	Add 003	Add 004

↳ 8 bits = 1 byte

1 bit is 0 or 1

Efficiency

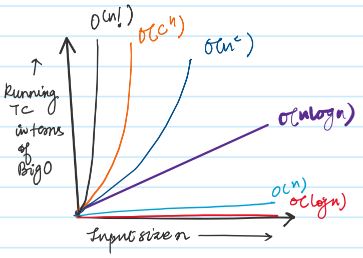
By counting fundamental operations → also referred to as Frequency count method.

Use Asymptotic analysis that helps measure ops

* Refer to Abdul Bari notes on TC + SC

→ Dont measure time but operation
→ expect performance to depend on size of input

Types: 1) Big O (upper bound)
2) Big Omega Ω (lower bound)
3) Theta (θ) – (Both-average)



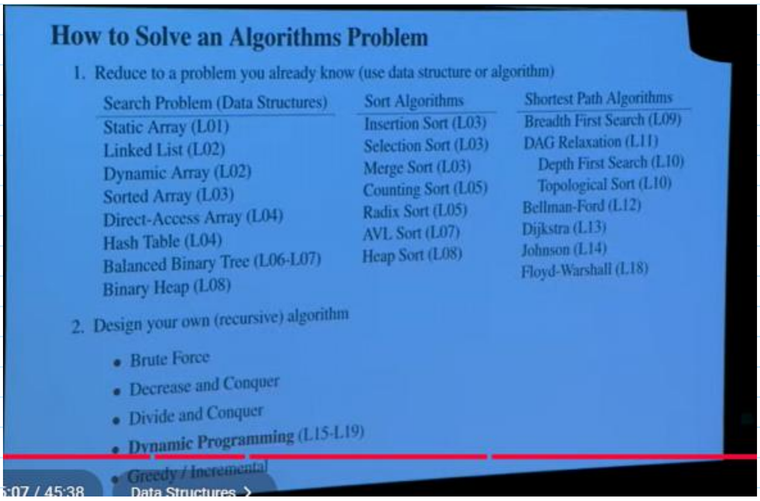
$O(1)\ O(\log n)\ O(n)\ O(n\log n)\ O(n^2)\ O(n^{ex})\ O(2^n)$

what about 64 bits in computers?
that means CPU has registers that can hold 64 bits = 8 bytes × 8 bits

8 bits	8	8	8	8	8	8	8

Byte 1 . . . .

$= 2^n$ distinct addresses
$= 2^{64}$



How to Solve an Algorithms Problem

1. Reduce to a problem you already know (use data structure or algorithm)

Search Problem (Data Structures)	Sort Algorithms	Shortest Path Algorithms
Static Array (L01)	Insertion Sort (L03)	Breadth First Search (L09)
Linked List (L02)	Selection Sort (L03)	DAG Relaxation (L11)
Dynamic Array (L02)	Merge Sort (L03)	Depth First Search (L10)
Sorted Array (L03)	Counting Sort (L05)	Topological Sort (L10)
Direct-Access Array (L04)	Radix Sort (L05)	Bellman-Ford (L12)
Hash Table (L04)	AVL Sort (L07)	Dijkstra (L13)
Balanced Binary Tree (L06-L07)	Heap Sort (L08)	Johnson (L14)
Binary Heap (L08)		Floyd-Warshall (L18)

2. Design your own (recursive) algorithm
   - Brute Force
   - Decrease and Conquer
   - Divide and Conquer
   - Dynamic Programming (L15-L19)
   - Greedy / Incremental

64 bit computer can have gigabytes of RAM with billion bytes
64 describe the width of certain processors operations, registers and address-related capabilities

↳ RAM capacity → storage available to hold data
memory address → a bit within the addressable space
64 bit register → inside CPU for operations
64 bit word → natural data unit → CPU's natural processing width
word size → unit of data that processor can handle in a single operation.
= native data size of the architecture = 64 bits

Lecture 2: Data structure & Dynamic Arrays

Three fundamental concepts:

1) Abstract data type (ADT)
what operations are available? Behaviour of data type
The contract in stack:
push(x), pop(), peek(), LIFO order

Standford course focus on them

↓

2) Data structure (Representation)
How is data organized & stored to support those operations?
eg: stack can be implemented using dynamic array or a linked list

→ linear
→ non linear

how elements reorganized

↓

3) Algorithm
what sequence of computational steps support those operations?

2 main interfaces:
1) ✓ Sequence
2) set

+ 2 main data structure approaches
1) Arrays
2) Pointers.

2 main interfaces :   + 2 main data structure approaches

1) ✓ Sequence   2) set   1) Arrays   2) Pointers.

↳ 1) static sequence interface

sequence of items $x_0, x_1, \dots x_{n-1}$

build(x) : $\Theta(n)$

len() : O(1) → python stores size of collection in the memory by default

iter_seq() : O(n)

get_at(i) : O(1)
set_at(i, x) : O(1) } as array address make it easy to access + word size is logn at least

which is [static arrays]

python has no static array

memory :

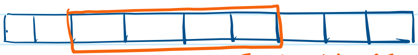
considering we have "word RAM" model

array is w-bit words + consecutive chunk

1 word size = logn at least why?

as to perform get at & set at in constant time machine should be able to represent any index i within single machine word

so the word size in modern computer = 64 bit ~ $\log_2(6)$



array[i] = memory[address(0) + i]

hence make access O(1)

↳ 2) Dynamic sequence :

[static sequence] +

1) insert_at(i, x) → shift all items post element at i th position
2) delete_at(i) → shift all to fill the space } O(n)

3) insert/delete at first or last — O(1) for last as space always exist in python — else O(1) for static array

O(n) for first as data needs to shift

Dynamic Seq operations :

Static Array

1) Insert and delete cost O(n) time

→ at last its O(1) in python else O(n) in all cases

Linked list

Insert & delete at front at O(1)

| x | | just need head address change

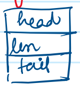
else O(n)

get + set → O(i) time

or O(n) as each node should be checked

Bad at random access as the position needs to be travelled every time

For last access save

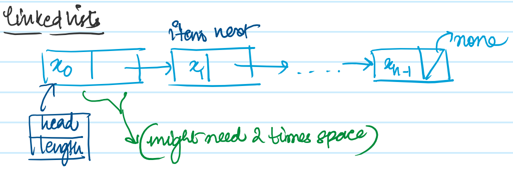


Data Structure Augmentation

– add new info to make things better

None of the above work efficiently, instead we use

Linked lists



(might need 2 times space)

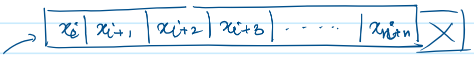
Solution → Dynamic array (Python lists)

→ Relaxes constraints on the size(array) = n (roughly n)

→ size of array $\Theta(n)$ & ≥ n

→ maintain $A[i] = x_i$

→ if we visualize :



Array

length = n (of items i stored)

size = (of the data structure Array with empty spaces)

① So assume we keep buffer & then extend it as the data grows!!

② But what if n = size?

allocate a new array of size := 2 × size

how will that help?

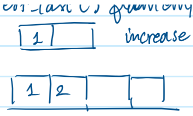
eg: we do insert_last() from empty array

pass1 : | 1 | | increase twice so size = 2

| 1 | 2 | | | size = 4

> resize cost = $\Theta(1+2+4+8+16+\dots+n)$

eg. [illegible]



pass1: increase twice so size = 2

size = 4

& so on size = 1, 2, 4, 8, 16
(2^0 for nothing)

$$\rightarrow \text{resize cost} = \theta(1+2+4+8+16+\dots+n)$$
$$= \theta\left(\sum_{i=1}^{\log n} 2^i\right) = \theta\left(2^{\log n}\right) = \theta(n) \approx \text{Amortized } O(1)$$

in geometric sequence, where seq is growing exponentially, we can simply pick the last item

Amortization

Operation takes $T(n)$ amortized time if any k operations take $\leq k\,T(n)$ time

or simply →
"averaging over operation sequence" ← Total time it takes to do n operations cannot take more than x time
— over the long term, this is on the average

Sequence Interface

Data Structures	Static	Dynamic		
	get_at(i) set_at(i, x)	insert_first(x) delete_first()	insert_last(x) delete_last()	insert_at(i, x) delete_at(i)
Array	1	n	n	n
Linked list	n	1	n	n
Dynamic Array	1	n	1 (a)	n

Lecture 3 (Introduction to Algorithms – Problem session 1 – Asymptotic Behavior of Functions & Double ended

$\sqrt{n} < n < n+\sqrt{n}$

$\log(n) = o(n^b)$

How big is n!
→ Stirling's approximation
$\sqrt{2\pi n}\left(\frac{n}{e}\right)^n$

Binomial coefficient

$\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$

no of n ways to do k

so, $\log(n!) = \log\left(\sqrt{2\pi n}\left(\frac{n}{e}\right)^n\right) \rightarrow \theta(n\log n)$

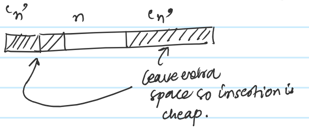
Implementation of an interface is array, linked list etc

Q. Swap_ends (Data structure) using data structure specific operations

```
swap_ends (arr) .
    x1 = arr.delete_first()
    x2 = arr.delete_last()

    arr.insert_first(x2)
    arr.insert_last(x1)
```

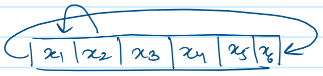
you could use temp and ~~simply~~ swap



Dynamic array has doubly ended space.
What about extra space? – At no point we should have extra space & waste it

"Keep some space always"

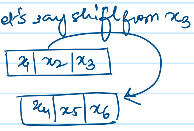
Q. shift left
shift_left(...) in $O(k)$



put x_1 behind x_6 and x_2 to x_1

Bruteforce
delete x_1 & insert at x_6
for n times?
x_2 is naturally at x_1 now?

let's say shift from x_3



so let's split the array & reverse the first half and stick it at the end?

Recursive?

```
if k==0 or k==1:
    return

x = delete_first()
insert_last(x)
shift_left(0, k-1)
```

← array indices